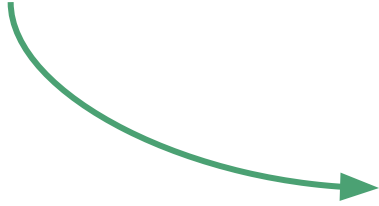


Computing GC content with for loops

BILD 62

Quick check in...



<https://www.menti.com/blhpa9c4jpki>

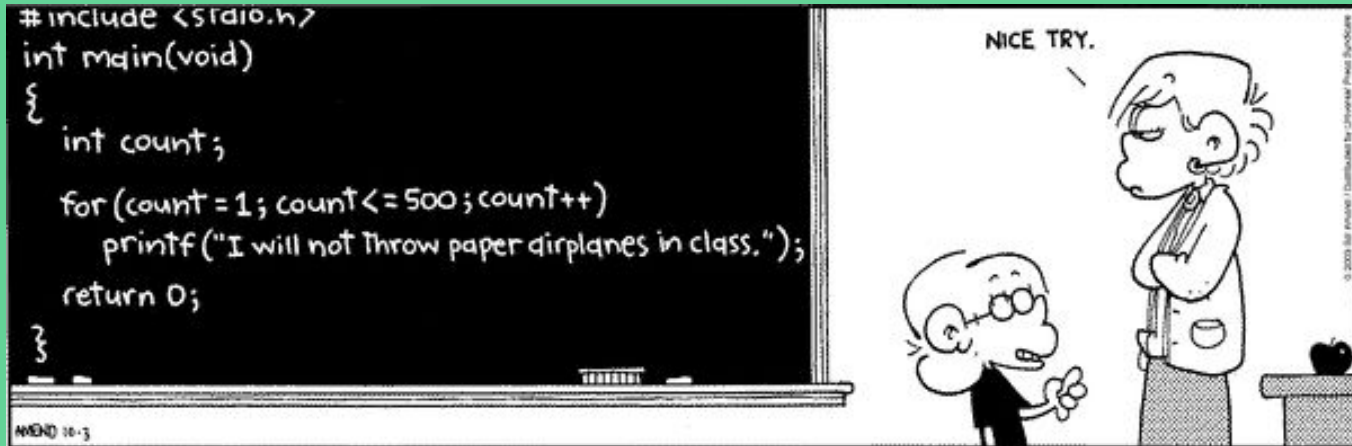
Recap of last time:

computing GC content with conditionals

- Can we do this with `elif` statements?
- Can we alert the user if the function gets incorrect input (a string of incorrect length)?

A **loop** is a procedure to repeat a piece of code.
(another way of saying this is that it **iterates** through code)

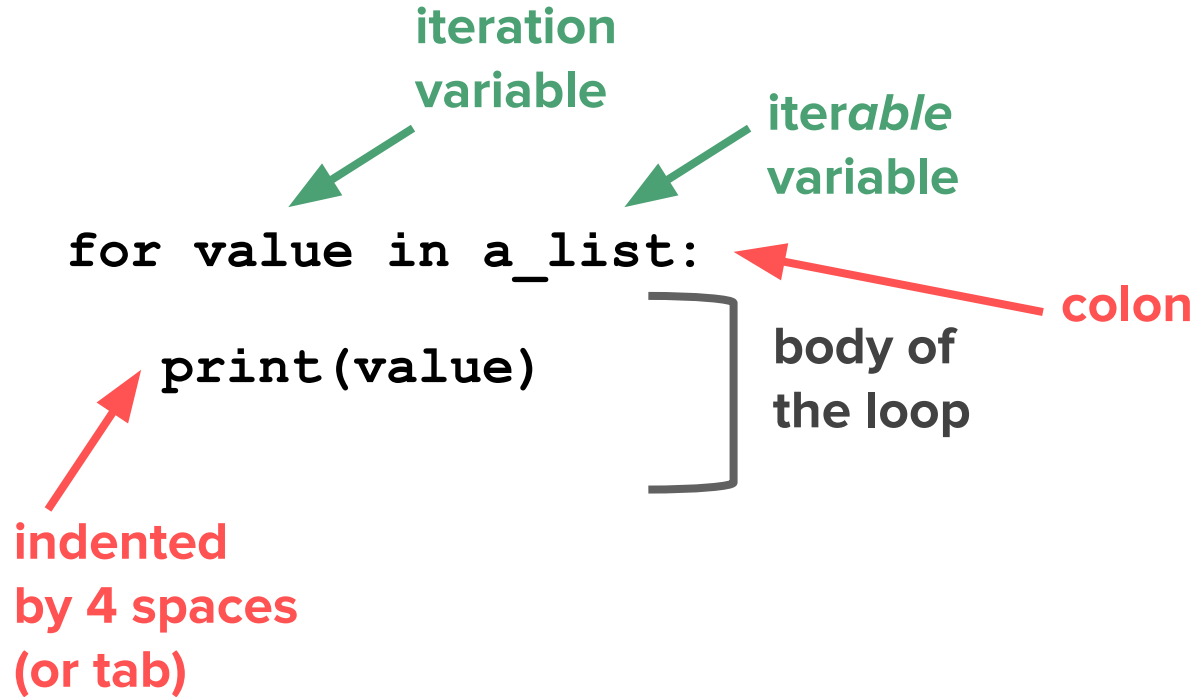
- Loops enable you to re-run blocks of code for as many times as you need.
- Python has two main ways to run loops: **for** & **while**



Objectives for this week

- Write a **for** loop to iterate over elements in an object
 - Use a **counter** in a loop
 - Use a **for** loop to count GC content and CCAAT boxes in a DNA string
 - Write **while** loops and implement **continue** and **break**
 - Develop a **growth mindset** towards your programming growth
-

for loop syntax



The diagram illustrates the syntax of a Python for loop. The code is shown as follows:

```
for value in a_list:  
    print(value)
```

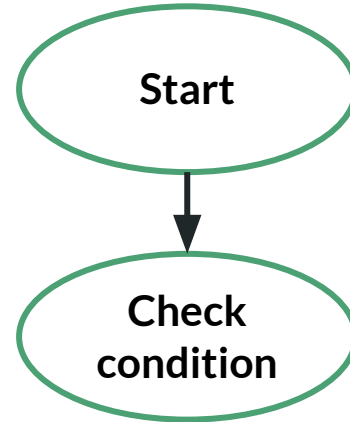
Annotations with arrows point to specific parts of the code:

- iteration variable**: Points to `value`.
- iterable variable**: Points to `a_list`.
- colon**: Points to the colon at the end of the first line.
- body of the loop**: Points to the indented line `print(value)`.
- indented by 4 spaces (or tab)**: Points to the indentation of the `print(value)` line.

A **for loop** is a procedure to repeat code for every element in a sequence.

for loop syntax

```
a_list = [1,2,3]  
  
for value in a_list:  
    print(value)
```



for loop syntax

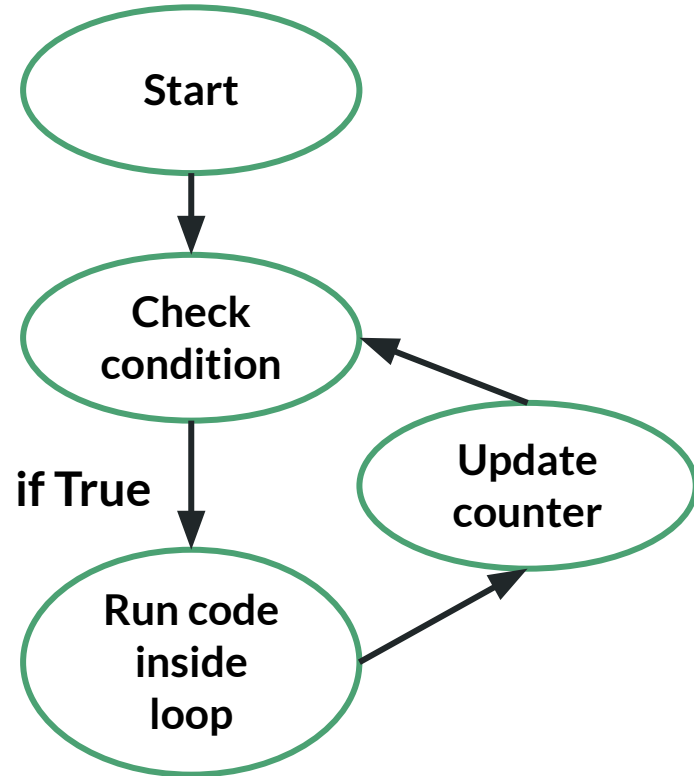
```
a_list = [1,2,3]
```

```
for value in a_list:
```

```
    print(value)
```

1

output

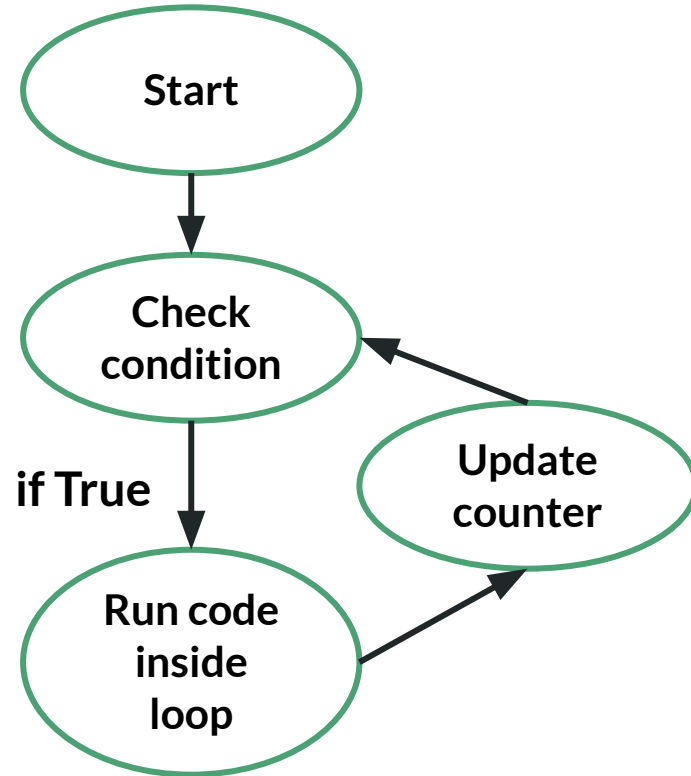


for loop syntax

```
a_list = [1,2,3]

for value in a_list:
    print(value)
```

```
1 |
2 | output
```

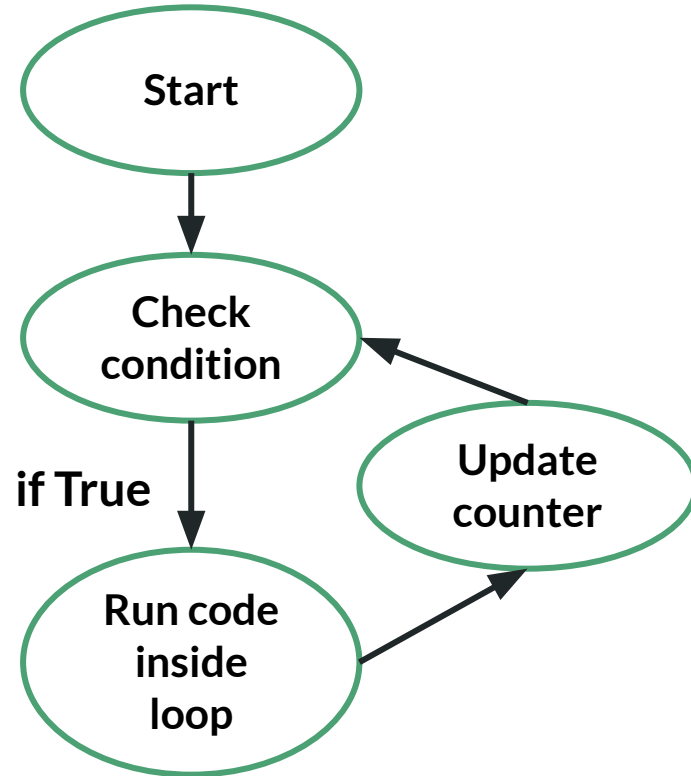


for loop syntax

```
a_list = [1,2,3]

for value in a_list:
    print(value)
```

```
1 |
2 | output
3 |
```



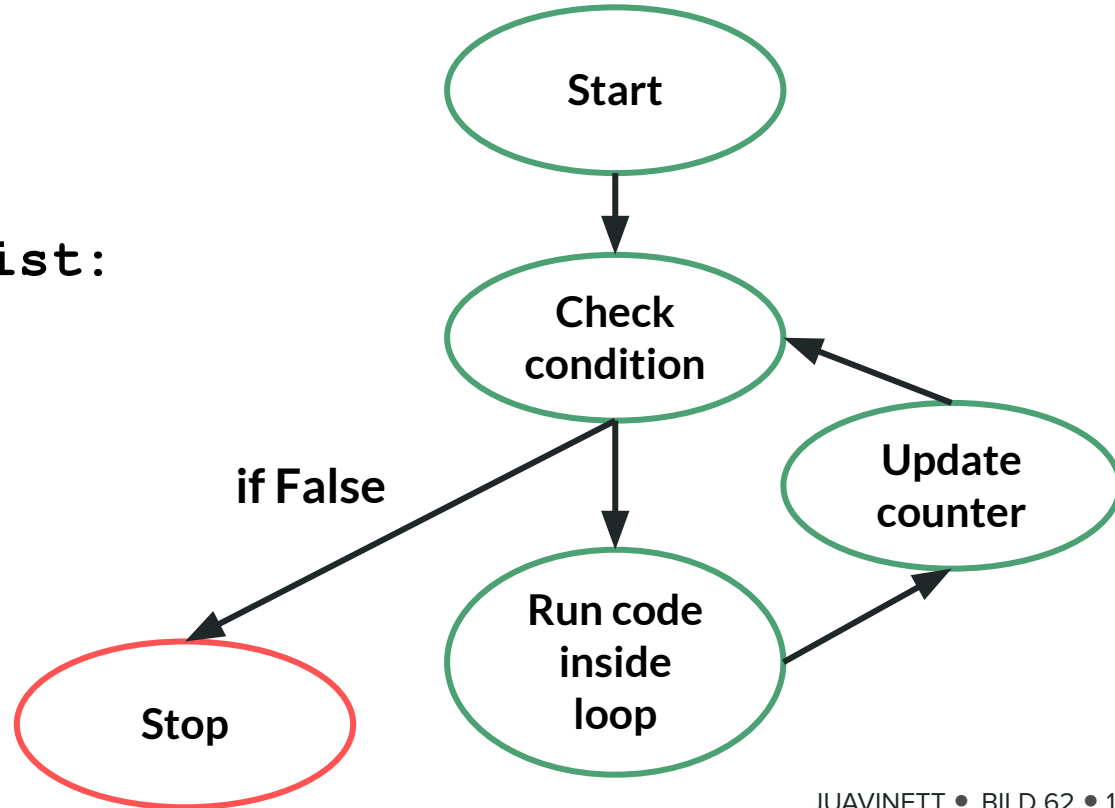
for loop syntax

```
a_list = [1,2,3]

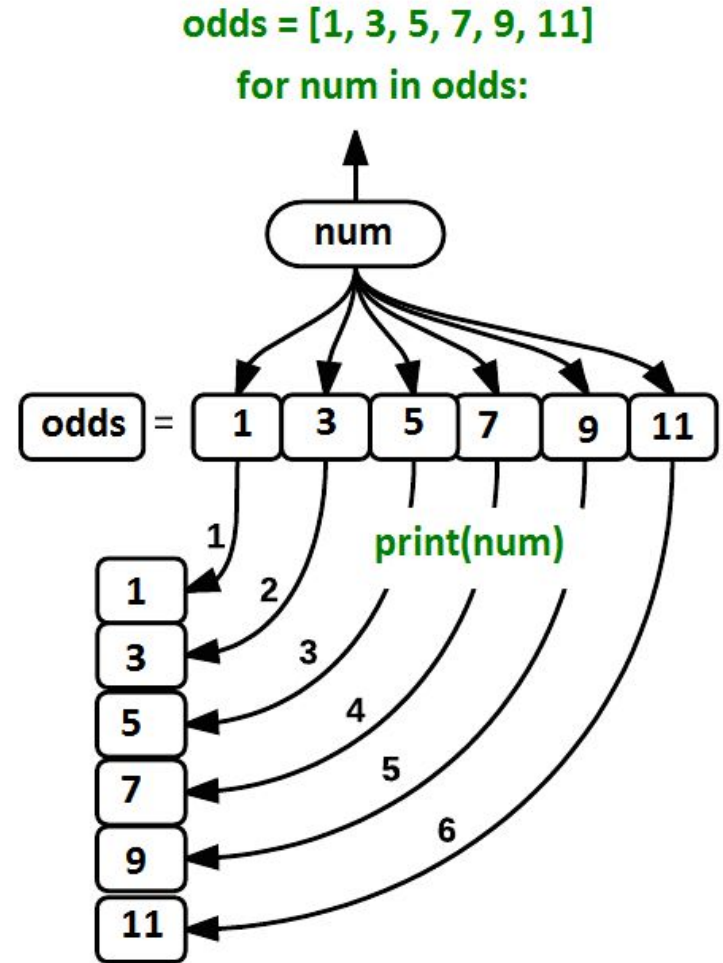
for value in a_list:

    print(value)
```

```
1 |
2 | output
3 |
```



Another way to visualize working
through a loop
([source](#))



efficiency benefit of `for` loops

Each of these would accomplish the same thing:

Option #1: 2+ lines of code

```
for value in a_list:  
    print(value)
```

**Option #2: as many lines of code
as there are list entries**

```
print(a_list[0])  
print(a_list[1])  
print(a_list[2])  
...
```

Second task: count the # of “CAT” boxes (CCAAT) in a string of DNA.

The “CAT” box generally appears near the spot where transcription begins!

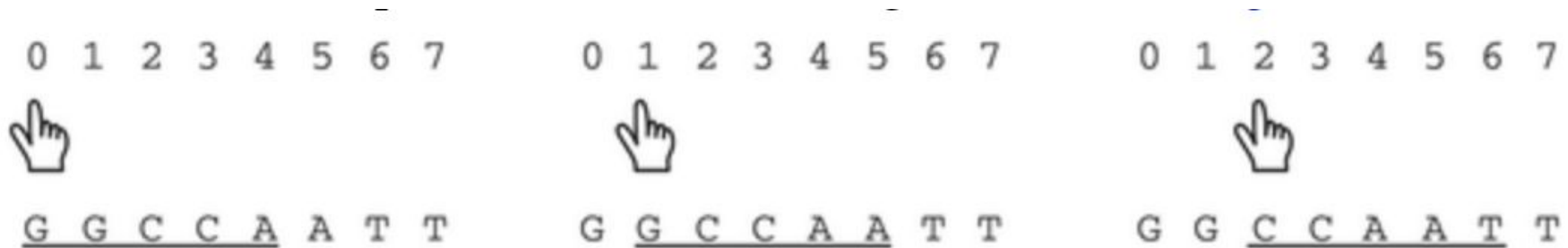


```
>>> countCCAAT( 'GGCCAATTGCCAAT' )  
>>> 2
```

We can also loop over a list of indices!

Let's say we want to look for a “CAT” box, a common motif in DNA, with the sequence CCAATT

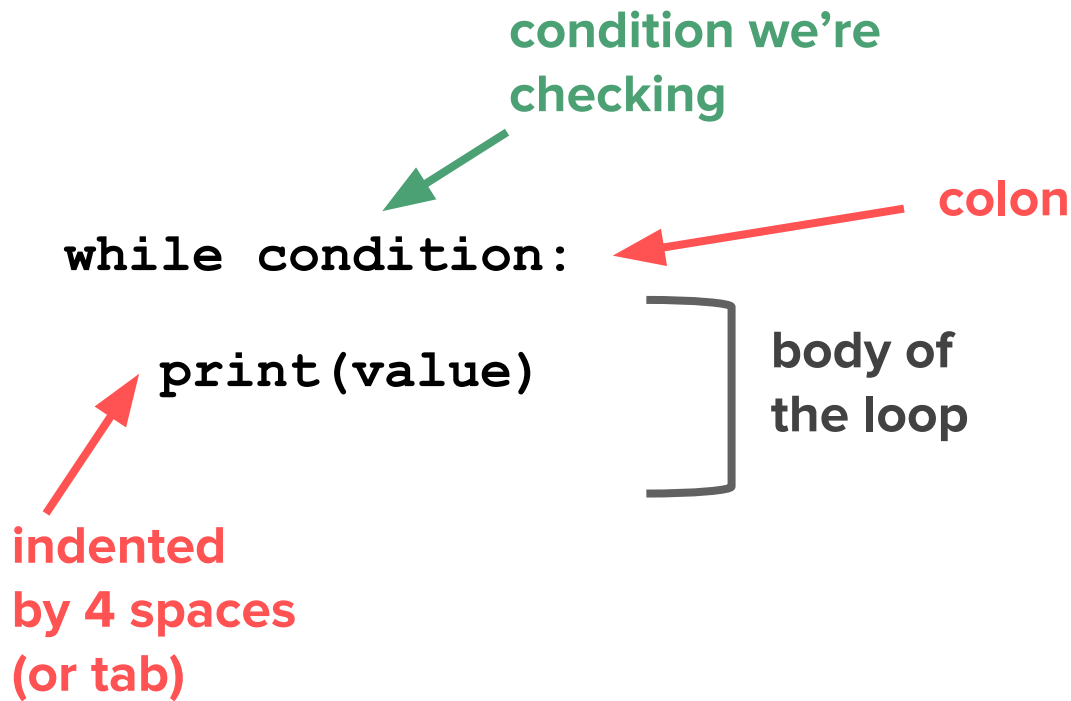
Since we want to look at a **slice** of DNA, rather than looping through individual items in the string, we need the indices.



Into the notebook...



while loop syntax



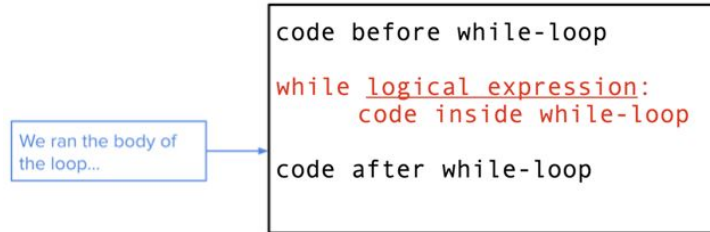
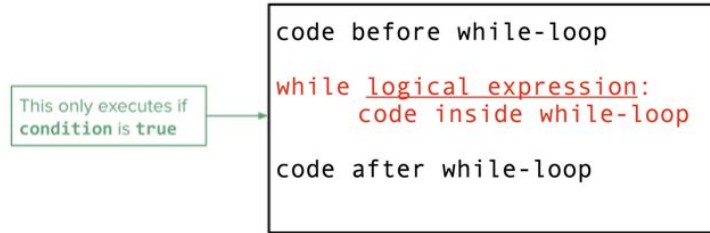
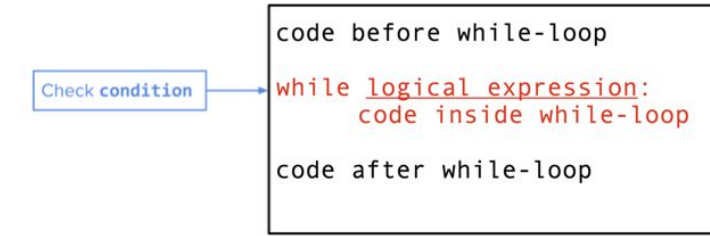
The diagram illustrates the syntax of a while loop with the following code and annotations:

```
while condition:  
    print(value)
```

- condition we're checking**: A green arrow points to the `condition` part of the `while` statement.
- colon**: A red arrow points to the colon (`:`) at the end of the `while` statement.
- body of the loop**: A bracket on the right side of the `print(value)` line indicates the code block that is repeated.
- indented by 4 spaces (or tab)**: A red arrow points to the indentation of the `print(value)` line.

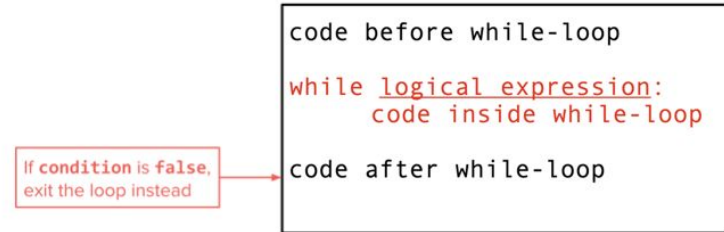
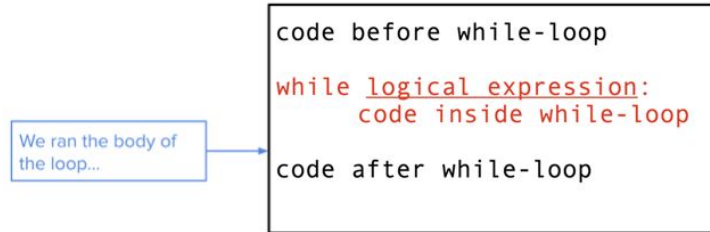
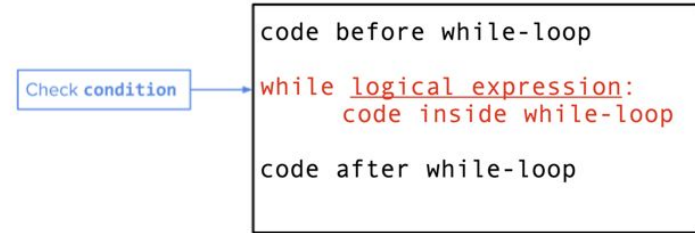
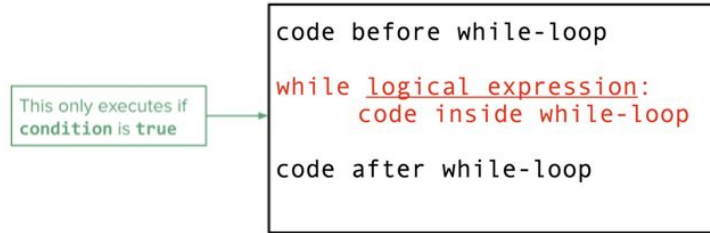
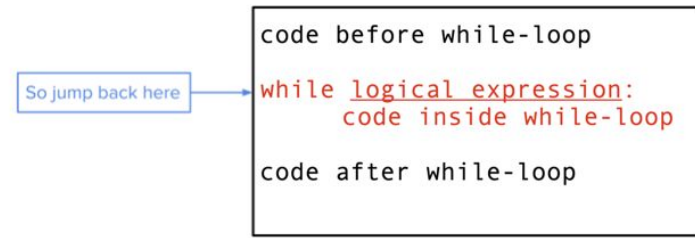
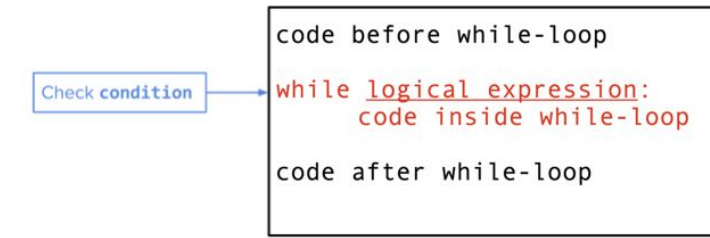
While this condition is true, the loop will run!

It will repeat until the condition is no longer True.



Order of execution in a while loop

(Image credit: Niema Moshiri & Sabeel Mansuri)



Order of execution in a while loop

(Image credit: Niema Moshiri & Sabeel Mansuri)

Topics from this lecture & corresponding notebook

- Syntax of **for** and **while** loops
- How to iterate through strings, lists, and dictionaries
- Using a counter to count loop iterations
- Looping over lists of indices
- Calling functions within functions
- Using **break** to interrupt a loop, and **continue** to skip a loop
- Functions we learned: **range()** , **enumerate()**

Resources

[Stepik Introduction to Python book, Chapter 3](#)

[Whirlwind Tour of Python: Control Flow](#)